

Day 03: USER MANUAL - 1

Secure Coding Practices

Android Application Security & Vulnerability Remediation

Table of Contents

Introduction.....	3
Before You Start: Where Things Live in Android Studio.....	3
1. Securing Components.....	4
1.1 Securing Application Components.....	4
2. Parameterized Queries.....	4
2.1 SQL Injection.....	4
2.2 Preventing Command Injection.....	4
3. Input Validation.....	5
CONTINUE THIS IN SESSION - 2.....	6
4. Exception Handling.....	6
5. Insecure/Secure Cryptography.....	7
6. Secure Storage.....	8

Introduction

With the Android platform fast becoming a target of malicious hackers, applications security is no longer an add-on, but a crucial part of the developer's job.

This post provides secure coding practices that a developer need to follow in order to design and implement robust, rugged, and secure apps for Android device.

Before You Start: Where Things Live in Android Studio

Folder / File	Contents / Purpose
app/manifests/AndroidManifest.xml	Exported components, permissions, and intent filters
app/java/MainActivity.java	All runtime logic
app/res/values/strings.xml	Permission labels
Gradle Scripts/build.gradle	SQLCipher and security-crypto libraries
Logcat panel (bottom toolbar)	Application logging and logs check
proguard-rules.pro (app root)	Rules to strip debug logs from release builds

1. Securing Components

Download Link for the Insecure application

[Download App](#)

1.1 Securing Application Components

Insecure Code :

```
<activity
    android:name=".WelcomeActivity"
    android:exported="true"
    android:windowSoftInputMode="adjustResize" />
```

You should add the following attribute to the application component's XML element In the AndroidManifest.xml File

SecureCode :

```
<activity
    android:name=".WelcomeActivity"
    android:exported="false"
    android:windowSoftInputMode="adjustResize" />
```

2. Parameterized Queries

2.1 SQL Injection

consider a case where the user supplies a username and password to an application, which must then verify that the combination is valid to allow access.

2.2 Preventing Command Injection

The unsafe way to form this statement looks something like this:

Insecure Code :

```
String query = "SELECT * FROM users WHERE username = " + user + " AND password = " + pass + """;
textStatus.setText("Executed Query:\n" + query + "\n");
Cursor cursor = db.rawQuery(query, null);
boolean loginMatched = cursor.getCount() > 0; cursor.close();
```

Here is the proper way to perform such a query against the database, where the command is separated from the data:

Secure Code:

IMPORT THE FOLLOWING AT THE TOP OF THE CLASSES WHERE THERE ARE OTHER IMPORTS:

```
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteStatement;
```

WRITE THIS CODE IN THE PLACE OF THE ABOVE INSECURE CODE

```
String userQuery = "SELECT COUNT(*) FROM users WHERE username = ? AND
password = ?";
SQLiteStatement prepStatement = db.compileStatement(userQuery);
prepStatement.bindString(1, user);
prepStatement.bindString(2, pass);
boolean loginMatched = prepStatement.simpleQueryForLong() > 0;
```

3. Input Validation

Never trust external data: Treat all data submitted from outside the app as potentially manipulated by an attacker.

Always validate input: Verify that all direct and indirect external data is safe to process as a primary secure coding practice.

Insecure Code :

There is no input validation code at all in the insecure app. We should write that part of the code.

The below example shows a revision to fix the security bugs:

SecureCode :

Declare this Variable in the Variable declaration Section:

```
private static final String EMAIL_REGEX =
"^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$";
```

Add this Part of code just above the SQL Injection Prevention Code that we have written previously:

```
if (!user.matches(EMAIL_REGEX)) {
    textStatus.setText("Input Validation Failed: Invalid email format.");
    return;}
}
```

Validating the safety of input data is called “Input Validation” and it is a fundamental secure

coding method.

CONTINUE THIS IN SESSION - 2

4. Exception Handling

- **Safe Error Recovery:** Handle exceptions so the app transitions to a safe state without leaking sensitive info through the UI or logs.
- **Prevent Runtime Errors:** Proactively use proper null and bounds checks to avoid standard runtime exceptions (e.g., `NullPointerException`).

Make sure that all confidential information handled by high-risk applications is always wiped during execution of the finally blocks.

Insecure Code :

INSIDE SaveNote()

```
// No try/catch anywhere in this app: an empty note makes charAt(0)
// throw StringIndexOutOfBoundsException, which is never caught, so
// the app crashes and closes.
char firstChar = note.charAt(0);
```

SecureCode :

WRITE THIS CODE IN THE PLACE OF THE ABOVE INSECURE CODE

```
try {
    char firstChar = note.charAt(0);
} catch (StringIndexOutOfBoundsException e) {
    textStatus.setText("Please write something in the note before saving.");
    Log.e("NOTE_ERROR", "Empty note.", e);
    return;
}
```

5. Insecure/Secure Cryptography

Protect user data by using strong cryptographic algorithms, unique salts, random Initialization Vectors (IVs), and proper key derivation parameters.

Insecure Code :

```
notePrefs.edit().putString("note_value", note).apply();

textStatus.setText("Saved Plaintext Note: " + note + "\n");
```

SecureCode :

IMPORT THE FOLLOWING AT THE TOP OF THE CLASSES WHERE THERE ARE OTHER IMPORTS

```
import java.security.SecureRandom;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;
```

Declare this Variables in the Variable declaration Section:

```
private byte[] salt;
private byte[] iv;
private static final String PBE_ALGORITHM = "PBEWithSHA256And256BitAES-CBC-BC";
private static final String CIPHER_ALGORITHM = "AES/CBC/PKCS5Padding";
private static final int NUM_OF_ITERATIONS = 1000;
private static final int KEY_SIZE = 256;
```

Inside the onCreate Method Just Above the button setOnClickListener() write this code:

```
salt = new byte[8];
new SecureRandom().nextBytes(salt);
iv = new byte[16];
new SecureRandom().nextBytes(iv);
```

WRITE THIS CODE IN THE PLACE OF THE ABOVE INSECURE CODE

```
String encryptedNote = encryptData(note);
notePrefs.edit().putString("note_value", encryptedNote).apply();
```

```
textStatus.setText("Note encrypted and saved.\n");
```

CREATE THE BELOW METHOD INSIDE THE CLASS

```
private String encryptData(String plainText) {
    try {
        PBEKeySpec pbeKeySpec = new PBEKeySpec("note-encryption-key".toCharArray(),
        salt, NUM_OF_ITERATIONS, KEY_SIZE);
        SecretKeyFactory keyFactory = SecretKeyFactory.getInstance(PBE_ALGORITHM);
        SecretKey secretKey = keyFactory.generateSecret(pbeKeySpec);
        SecretKeySpec secretKeySpec = new SecretKeySpec(secretKey.getEncoded(), "AES");
        IvParameterSpec ivSpec = new IvParameterSpec(iv);

        Cipher encCipher = Cipher.getInstance(CIPHER_ALGORITHM);
        encCipher.init(Cipher.ENCRYPT_MODE, secretKeySpec, ivSpec);
        byte[] encryptedBytes = encCipher.doFinal(plainText.getBytes());

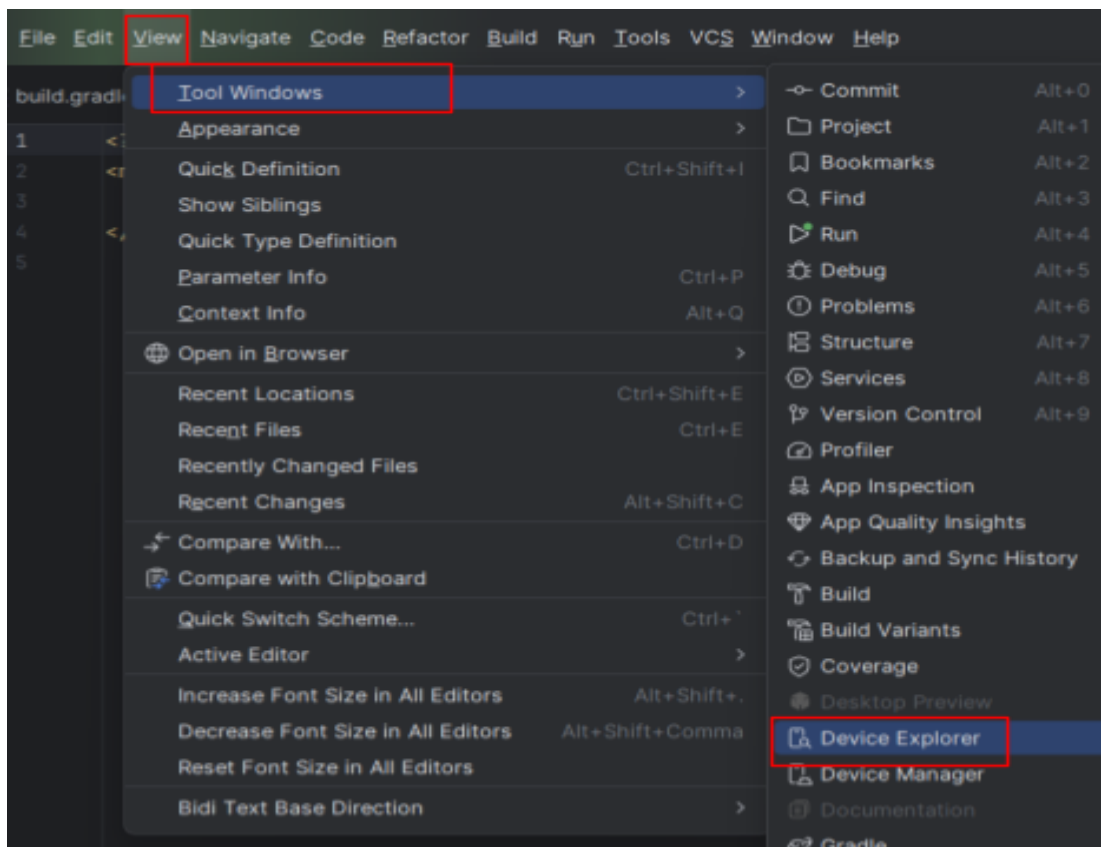
        return android.util.Base64.encodeToString(encryptedBytes,
        android.util.Base64.DEFAULT);
    } catch (Exception e) {
        throw new RuntimeException("Encryption failed", e);
    }
}
```

6. Secure Storage

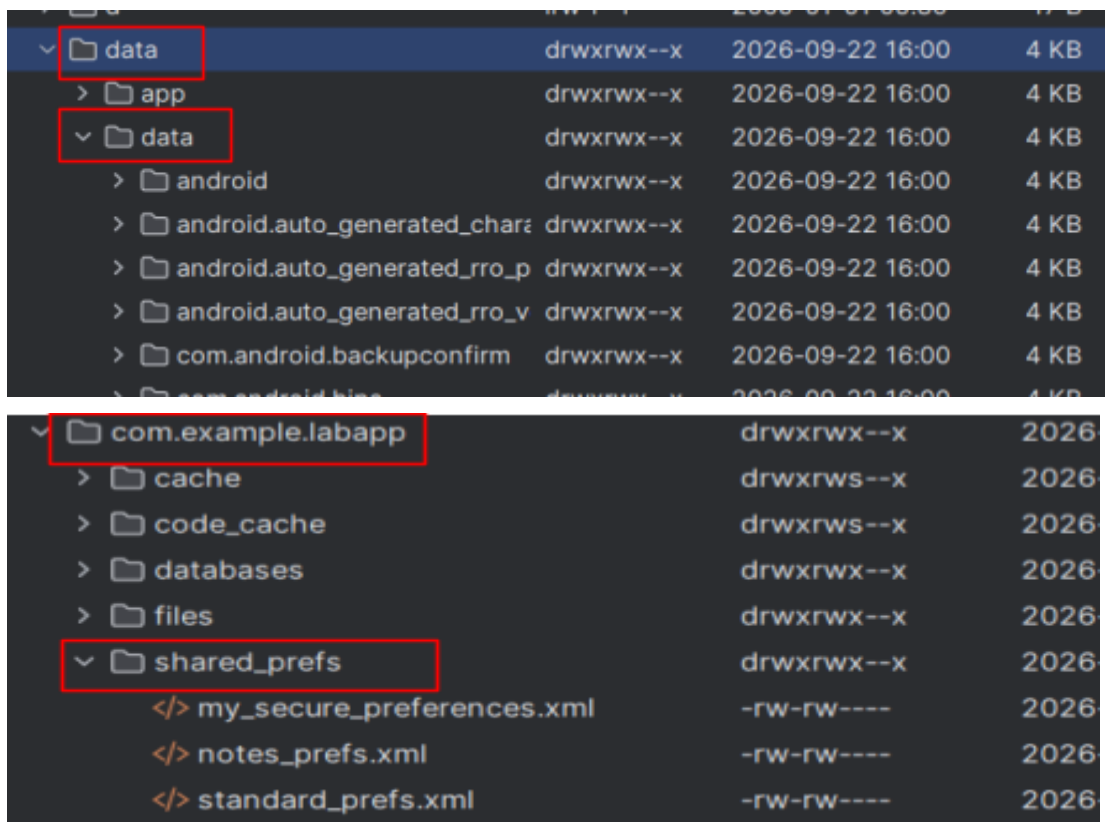
Android provides a simple framework for app developers to persistently store key-value pairs of primitive data types.

It is a library that wraps the default Android SharedPreferences to encrypt the key-value pairs for protecting them against attackers.

To See the shared preferences data, follow below steps as shown in screenshots



This will be opened below the emulator after clicking on Device Explorer.



Insecure Code :

```
SharedPreferences prefs = getSharedPreferences("standard_prefs", MODE_PRIVATE);
prefs.edit().putString("saved_user", user).putString("saved_pass", pass).apply();
```

SecureCode :

IMPORT THE FOLLOWING AT THE TOP OF THE CLASSES WHERE THERE ARE OTHER IMPORTS

```
import androidx.security.crypto.EncryptedSharedPreferences;
import androidx.security.crypto.MasterKey;
```

WRITE THIS CODE IN THE PLACE OF THE ABOVE INSECURE CODE

```
try {
    MasterKey masterKey = new MasterKey.Builder(this,
MasterKey.DEFAULT_MASTER_KEY_ALIAS)
        .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
        .build();

    SharedPreferences prefs = EncryptedSharedPreferences.create(
        this,
        "my_secure_preferences",
        masterKey,
        EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
        EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM);

    prefs.edit()
        .putString("secure_user", user)
        .putString("secure_pass", pass)
        .apply();
} catch (Exception e) {
    Log.e("STORAGE_ERROR", "Failed to save credentials securely.", e);
}
```